

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский технический университет связи и информатики»**

Кафедра «Программная инженерия»

Лабораторная работа №1
по дисциплине «Структуры и алгоритмы обработки данных»
по теме «Быстрый доступ к данным»

Выполнил студент группы:
БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Борзов В.М.

Москва, 2026

Оглавление

Введение	3
Выполнение	4
Задание 1. Создание проекта	4
Задание 2. Доступ по индексу и линейный поиск	4
Задание 3. Доступ по ключу	6
Задание 4. Исследование функции hash()	6
Задание 5. Простая хеш-таблица методом цепочек	6
Задание 6. Коэффициент заполнения и коллизии	7
Задание 7. Представление бинарной кучи	7
Задание 8. Реализация sift up и вставки	7
Задание 9. Реализация sift down и извлечения корня	8
Задание 10. Приоритетная очередь	8
Задание 11. Тор-К с полной сортировкой	8
Задание 12. Тор-К с помощью кучи	8
Задание 13. Самостоятельная работа	8
Вывод	9
Контрольные вопросы	10

Введение

Целью данной лабораторной работы является изучение влияния способа организации данных на скорость выполнения основных операций, освоение доступа к данным по индексу и по ключу, понимание принцип работы хеш-таблиц и причины возникновения коллизий, обучение работы с бинарной кучей и приоритетной очередью, а также реализация поиск Top-K элементов без полной сортировки данных.

В ходе выполнения лабораторной работы необходимо:

- 1) сравнить стоимость различных способов доступа к данным;
- 2) реализовать линейный поиск и простую хеш-таблицу с обработкой коллизий;
- 3) исследовать влияние коэффициента заполнения;
- 4) самостоятельно реализовать операции sift up и sift down для бинарной кучи;
- 5) сравнить два способа поиска нескольких наибольших элементов.

Выполнение

Задание 1. Создание проекта

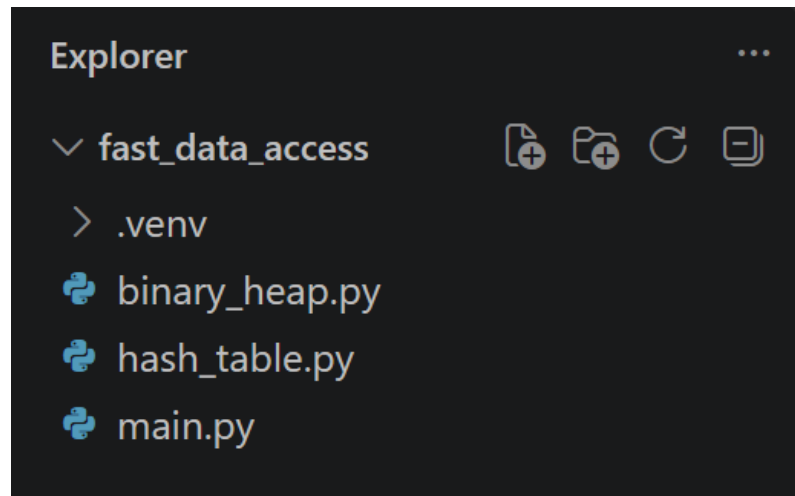


Рисунок 1 — Структура проекта

Задание 2. Доступ по индексу и линейный поиск

```
1 import timeit as tmt
2 # from time import perf_counter
3 from random import randint
4 import heapq as hp
5 from hash_table import HashTable
6 from binary_heap import MinHeap
7
8 big_num = int(1e5) # >=1e5
9 a = int(1e2)
10
11 def get_time(func, n=big_num):
12     timer = tmt.Timer()
13     total = tmt.timer.timeit(number=n)
14     return total
15
16 # 2
17 def task2():
18     def linear_search(numbers, target):
19         for i in range(len(numbers)):
20             if numbers[i] == target:
21                 return i
22         return -1
23
24     array_int = [randint(-a, a) for i in range(big_num)]
25
26 # 3
27 def task3():
28     def id_search(values, target):
29         for val in values:
30             if val[0] == target:
31                 return target
```

```

32         return -1
33
34     array_dict = [(i, randint(-a, a)) for i in range(big_num)]
35     st = 0
36     md = big_num // 2
37     fn = big_num - 1
38     id_search(array_dict, st)
39     id_search(array_dict, md)
40     id_search(array_dict, fn)
41
42     new_dict = dict(array_dict)
43
44 # 4
45 def task4():
46     print('hash({})_=_{},_hash({})_=_{},_hash({})_=_{}\nhash({})_=_{},_
47           hash({})_=_{}'.format(
48         0, hash(0),
49         5, hash(5),
50         -5, hash(-5),
51         'aaaaaa', hash('aaaaaa'),
52         'uwu', hash('uwu')
53     ))
54
55     def bucket_index(key, table_size):
56         return hash(key) % table_size
57
58 # 5
59 def task5():
60     pass
61
62 # 6
63 def task6():
64     pass
65
66 # 7
67 def task7():
68     array = [2, 5, 7, 9, 11, 10, 15]
69     heap = MinHeap(array)
70
71 # 8
72 def task8():
73     pass
74
75 # 9
76 def task9():
77     pass
78
79 # 10
80 def task10():
81     heap = []
82     hp.heapify(heap)
83
84 # 11
85 def task11():
86     array_int = [randint(-a, a) for i in range(big_num)]
87     for K in [10, 100, 1000]:
88         pass
89
90 # 12

```

```

90 def task12():
91     def top_k_heap(data):
92         pass
93
94 # 13
95 def task13():
96     contestants = []
97
98 if __name__ == "__main__":
99     task2()
100    task3()
101    task4()
102    task5()
103    task6()
104    task7()
105    task8()
106    task9()
107    task10()
108    task11()
109    task12()
110    task13()

```

Задание 3. Доступ по ключу

Задание 4. Исследование функции hash()

Задание 5. Простая хеш-таблица методом цепочек

```

1 class HashTable():
2     def __init__(self):
3         self.bucket_num = 0
4         self.empty_array_list = []
5         self.elem_counter = 0
6
7     def set(self, key, value):
8         pass
9
10    def get(self, key):
11        pass
12
13    def remove(key):
14        pass
15
16    def load_factor(self):
17        if self.bucket_num > 0:
18            return self.elem_counter / self.bucket_num
19        return 0
20
21    def collision_count(self):
22        pass

```

Задание 6. Коэффициент заполнения и коллизии

Задание 7. Представление бинарной кучи

Задание 8. Реализация sift up и вставки

```
1 class MinHeap():
2     def __init__(self, args=None):
3         self.data = list(args) if args else []
4         if self.data:
5             for i in range(len(self.data) // 2 - 1, 1, 1):
6                 self._sift_down(i)
7
8     def get_parent(self, index):
9         return (index - 1) // 2
10
11    def get_left(self, index):
12        return 2 * index + 1
13
14    def get_right(self, index):
15        return 2 * index + 2
16
17    def _sift_up(self, index):
18        while index > 0:
19            parent = self.get_parent(index)
20            if self.data[parent] <= self.data[index]:
21                break
22            self.data[index], self.data[parent] = self.data[parent],
23                self.data[index]
24            index = parent
25        print(self.data)
26
27    def _sift_down(self, index):
28        size = len(self.data)
29        while True:
30            left = self.get_left(index)
31            right = self.get_right(index)
32            smallest = index
33            if left < size and self.data[left] < self.data[smallest]:
34                smallest = left
35            if right < size and self.data[right] < self.data[smallest]:
36                smallest = right
37            if smallest == index:
38                break
39            self.data[index], self.data[smallest] = self.data[smallest],
40                self.data[index]
41            index = smallest
42        print(self.data)
43
44    def pop(self):
45        if not self.data:
46            print('error: empty heap')
47        else:
48            root = self.data[0]
49            last = self.data.pop()
50            if self.data:
51                self.data[0] = last
52                self._sift_down(0)
```

Задание 9. Реализация sift down и извлечения корня

Задание 10. Приоритетная очередь

Задание 11. Тор-К с полной сортировкой

Задание 12. Тор-К с помощью кучи

Задание 13. Самостоятельная работа

Вывод

В ходе лабораторной работы: объяснять, почему способ организации данных влияет на скорость выполнения операций; различать доступ по известному индексу и последовательный поиск значения; использовать словарь для доступа по ключу и объяснять его связь с хеш-таблицей; объяснять назначение хеш-функции и причину возникновения коллизий; реализовывать простую хеш-таблицу методом цепочек; вычислять коэффициент заполнения хеш-таблицы и анализировать его влияние; представлять бинарную кучу в массиве и вычислять индексы родителя и потомков; реализовывать операции sift up и sift down; использовать бинарную кучу как основу приоритетной очереди; объяснять различие между полной сортировкой и поиском Тор-К; оценивать сложность основных операций и выбирать структуру данных исходя из требуемых операций.

Контрольные вопросы

- 1) *Почему выбор структуры данных влияет на скорость программы?*
Скорость операций поиска, добавления и удаления во многом зависит от того, как именно организован доступ к элементам в структуре данных.
- 2) *Что означает асимптотическая сложность $O(1)$?*
Оценка $O(1)$ означает, что количество основных шагов алгоритма не растёт пропорционально размеру набора данных.
- 3) *Почему доступ к элементу списка по известному индексу считается операцией $O(1)$?*
В списке Python элементы располагаются в определенном порядке и имеют индексы. Если индекс известен, доступ к элементу выполняется напрямую.
- 4) *Как работает линейный поиск и какова его сложность в худшем случае?*
Простейший линейный поиск последовательно сравнивает элементы с искомым значением. В худшем случае алгоритму приходится просмотреть весь список, следовательно сложность $O(n)$.
- 5) *Чем доступ по индексу отличается от доступа по ключу?*
Поиск по ключу реализован на основе хеш-таблицы в среднем выполняется значительно быстрее, чем последовательный. Но из-за дополнительной памяти и вычисления хеша ключа его внутреннее устройство сложнее.
- 6) *Для каких задач удобно использовать словарь `dict`?*
Если имеется у набора объектов имеются некие индивидуальные ключи.
- 7) *Что такое хеш-функция?*
Функция, реализующая алгоритм хэширования – способа преобразования ключа в числовое значение, которое используется для определения позиции элемента в хеш-таблице.
- 8) *Для чего хеш ключа преобразуется в индекс корзины?*
- 9) *Что называется коллизией в хеш-таблице?*
Коллизией называется ситуация, когда разные ключи после применения хеш-функции попадают в одну и ту же позицию таблицы.
- 10) *Почему коллизии нельзя считать ошибкой реализации?*
Коллизии являются нормальным свойством хеш-таблиц, и полностью исключить их для произвольного множества ключей обычно невозможно.
- 11) *В чём заключается метод цепочек?*
Каждая ячейка таблицы хранит не один элемент, а список пар ключ–значение.

Для поиска элемента сначала вычисляется индекс корзины, после чего просматриваются только элементы соответствующей цепочки.

- 12) *Что показывает коэффициент заполнения хеш-таблицы?*
Отношение числа сохраненных элементов к количеству корзин.
- 13) *Как увеличение коэффициента заполнения может повлиять на поиск?*
По мере увеличения коэффициента заполнения вероятность длинных цепочек возрастает, и поиск начинает требовать больше сравнений.
- 14) *Что такое бинарная куча?*
- 15) *Какое свойство выполняется в *min-heap*?*
Структура данных, представляющая собой почти полное бинарное дерево и обычно хранящаяся в обычном массиве
- 16) *Почему минимальный элемент *min-heap* всегда находится в корне?*
Так как значение каждого родителя не больше значений его потомков.
- 17) *Как вычислить индекс родителя элемента кучи?*
 $p = (i - 1) // 2$.
- 18) *Как вычислить индексы левого и правого потомков?*
 $l = 2 * i + 1; r = 2 * i + 2$.
- 19) *Для чего используется операция *sift up*?*
Для восстановления свойства кучи после добавления нового элемента (т.к. он сначала помещается в конец массива) его последовательно сравнивают с родителем и при необходимости меняют их местами.
- 20) *Для чего используется операция *sift down*?*
Для восстановления свойства кучи после извлечения корневого элемента (т.к. на его место переносится последний элемент массива) его замену последовательно сравнивают с потомками и при необходимости опускают вниз.
- 21) *Какова сложность вставки элемента в бинарную кучу?*
Сложность составляет $O(\log_N)$.
- 22) *Какова сложность извлечения корня бинарной кучи?*
Сложность составляет $O(\log_N)$.
- 23) *Что такое приоритетная очередь?*
Структура данных, где у каждого добавленного элемента есть свой уровень приоритета, от которого зависит порядок обработки.

- 24) *Чем приоритетная очередь отличается от обычной очереди FIFO?*
Обычная очередь обрабатывает элементы в порядке поступления. В приоритетной очереди следующим извлекается элемент с наиболее высоким приоритетом, независимо от того, когда он был добавлен.
- 25) *Как модуль `heapq` интерпретирует вершину кучи?*
В стандартном модуле `heapq` первым извлекается элемент с наименьшим значением. Если приоритет задается числом, необходимо заранее определить, означает ли меньшее число более высокий приоритет.
- 26) *Что означает задача Тор-К?*
Во многих задачах требуется найти не полный отсортированный набор, а только K лучших элементов.
- 27) *Почему для поиска Тор-К не всегда требуется сортировать весь набор данных?*
Куча гарантирует только положение корневого элемента и выполнение свойства кучи.
- 28) *Как работает алгоритм Тор-К с `min-heap` размером K ?*
Если K значительно меньше n , можно поддерживать `min-heap` размером K . После обработки всех данных в куче останутся K наибольших значений.
- 29) *Какова сложность Тор-К через кучу размера K ?*
Сложность составляет $O(n \log_K)$, т.к. для каждого нового элемента выполняется не более одной операции над кучей размера K .
- 30) *Почему элементы внутри бинарной кучи не обязаны быть полностью отсортированы?*
Бинарная куча по определению имеет определенный порядок хранения элементов.
- 31) *В каком случае полная сортировка может быть проще и оправданнее, чем поддержание кучи?*
При малом количестве элементов поддержание кучи имеет большую сложность, чем полная сортировка.
- 32) *Какие компромиссы между скоростью и памятью возникают при использовании хеш-таблиц?*
Быстрый доступ достигается за счет дополнительной памяти и вычисления хеша ключа.